DeepSeek Harness
@DeepSeekHarness · November 1, 2026

X ARTICLE

Exploring Super-agent Design: Towards a More Autonomous Agent System



> *The original article is in Chinese. This is a machine translation.*

> *This article discusses architectural ideas. It is not an introduction to existing features of DeepSeek Harness.*

1. Super-agent

Over the past year, coding agents and other single-task agents have shown that models can use tools to solve complex problems while needing less and less continuous monitoring and step-by-step intervention from humans. Models have also become better at coordinating multiple agents, which makes it practical to organize and schedule work across sessions. As people take less part in low-level execution, their role is gradually moving up to higher-level decisions and sign-off.

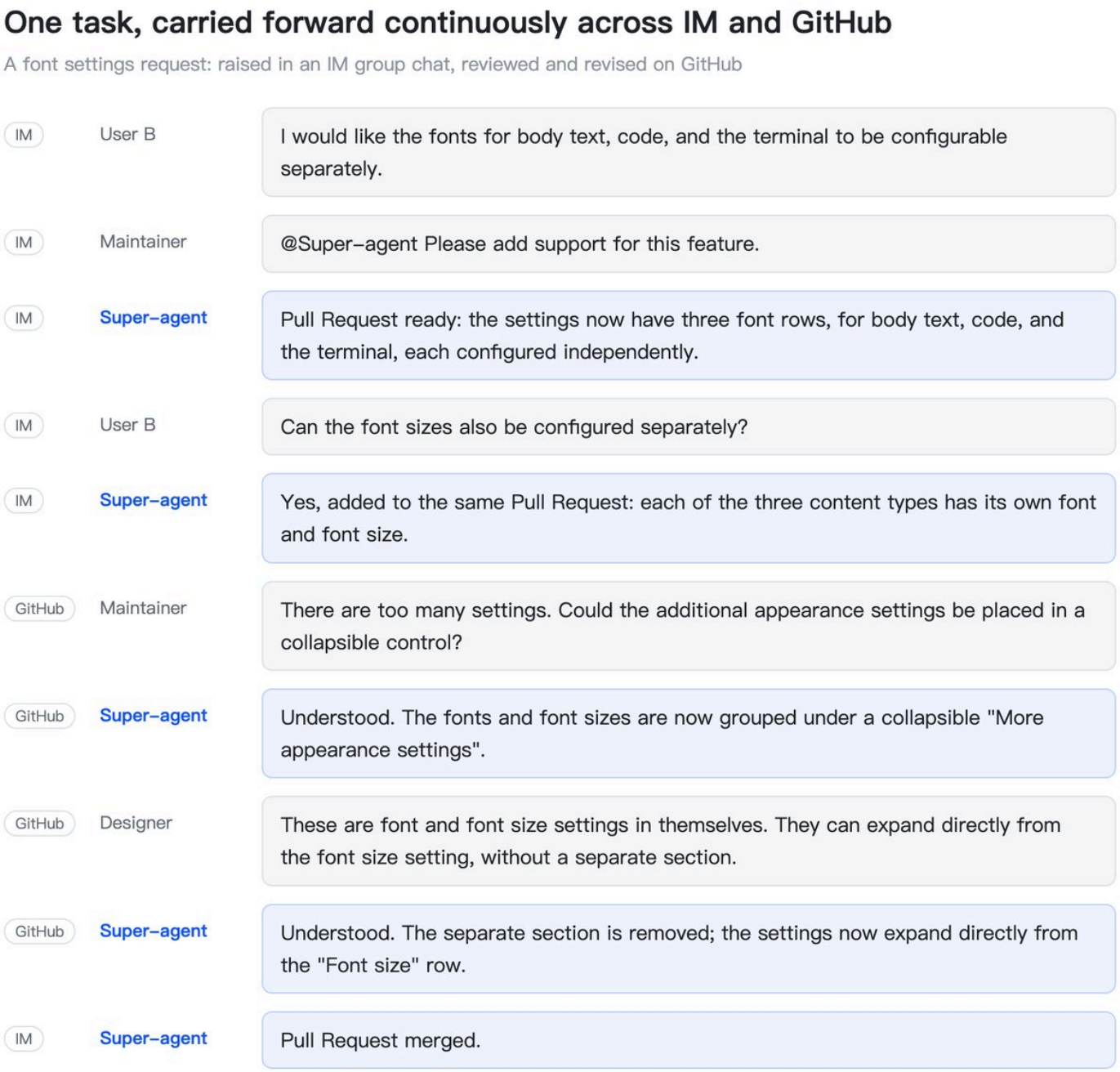
Better models have also enabled new applications. Recently, many personal agents have appeared. They connect directly to everyday messaging, calendar, email, and work platforms, monitor the feeds, and help with tasks. Personal agents are only one application of this always-online, event-driven way of working. This way of working is not specific to personal use and is also valuable in commercial work such as software development.

However, today's mainstream agent designs struggle to support this way of working across many tasks or a whole team. Agents still use a single session as the basic unit of work, and sessions are isolated from one another. A user running several tasks in parallel has to switch between sessions by hand and remember what each one contains. In our experience, a person can usually follow only 3 to 5 active sessions at once. With these designs, a single task is limited by model capability, but multi-tasking is limited by human attention.

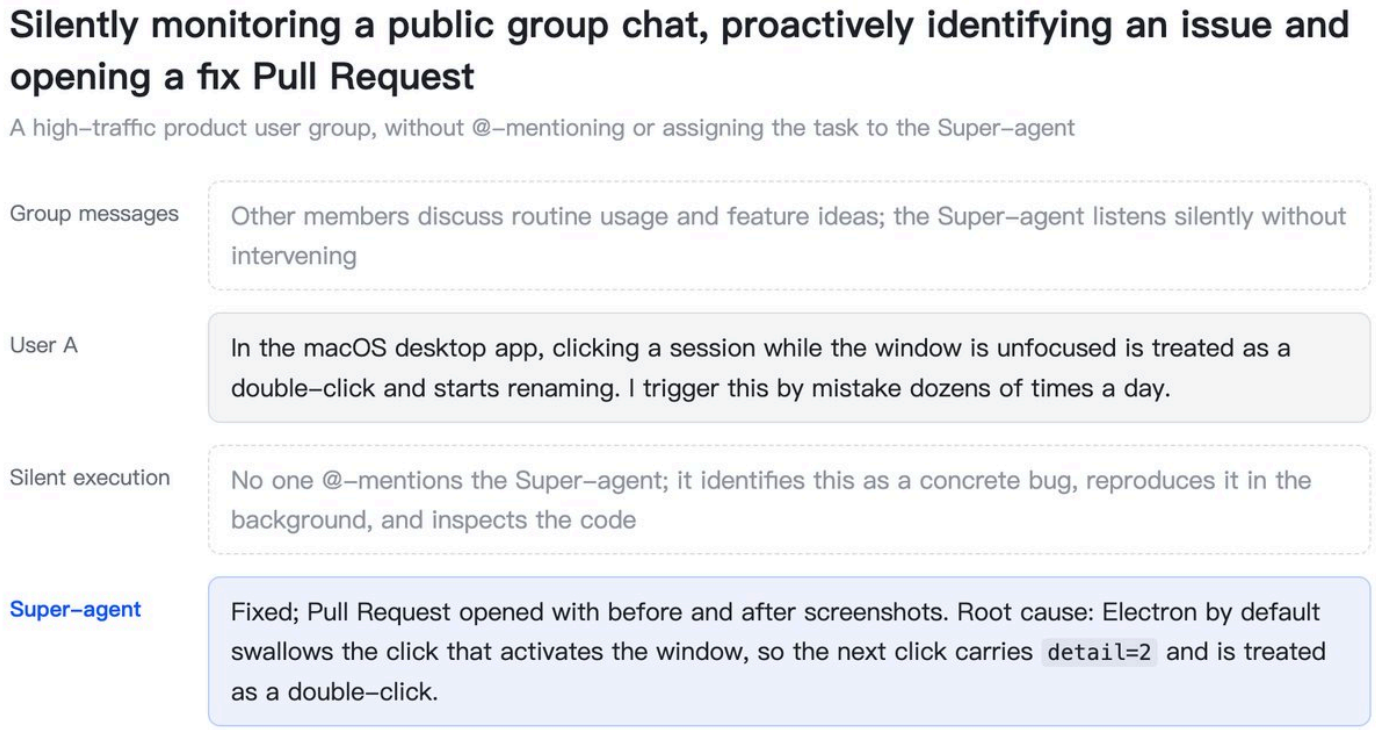
Adding CLIs, MCP, or external connectors to an existing local single-session agent lets it receive external messages, but its basic limitations remain. It still runs in one synchronous session that blocks on long operations, contexts from different tasks get mixed together, and everything stops whenever the local device goes offline. Keeping such a setup running takes a lot of manual effort.

Getting past these limits means decoupling the agent from any single, manually driven session. Instead, the agent should be one continuous entity that keeps its context across sessions and works on several complex tasks at once, without the tasks interfering with each other. We call this kind of agent and its system architecture a **Super-agent**.

For example, our internal Super-agent instance, which runs on DeepSeek Harness, can continue the same development task across IM and GitHub. It also monitors feedback in public group chats and, even when no one @-mentions it, finds root causes on its own and opens Pull Requests to fix them:



The same task continues across an IM group chat and a GitHub Pull Request review



The agent monitors a public group chat and, without being @-mentioned, finds a bug and opens a Pull Request to fix it

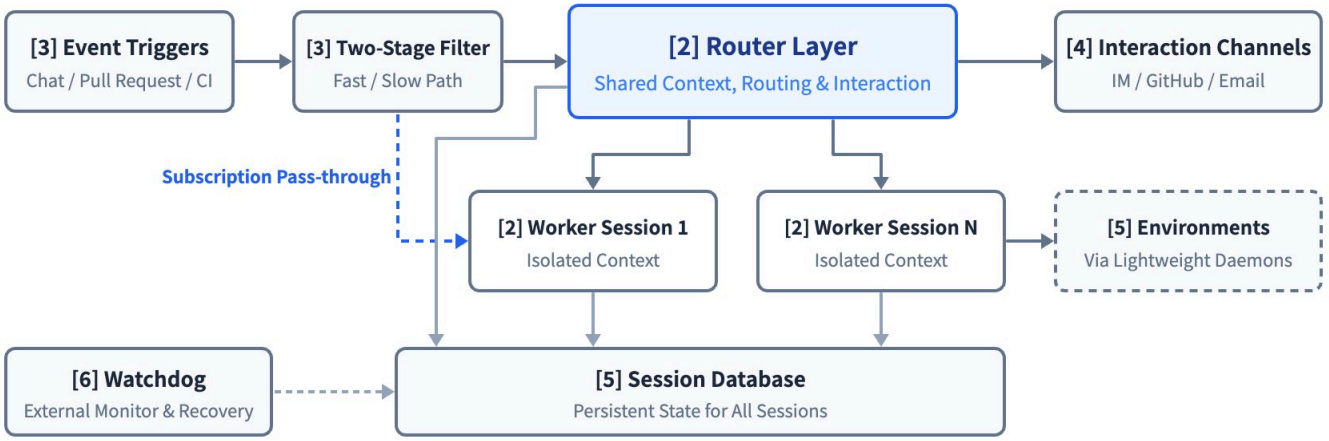
We believe a Super-agent has four basic characteristics:

- **Continuous:** Users can interact with it directly where they already work, such as IM chats and GitHub comments, and it always feels like one agent. They never have to wonder whether session X has information Y in its context. For example, if a user asks for Plan A in IM and later switches to Plan B in a GitHub Pull Request comment, the agent understands the change and adjusts what it is doing.
- **Productive:** It responds quickly and completes several complex tasks in parallel, each to a high standard. It can take the next instruction before the previous command or task has finished.
- **Proactive:** It does not wait for step-by-step instructions. It monitors events in its environment, acts on anything worth handling, and stays quiet otherwise.
- **Persistent:** It is always online, connects directly to the user's real working environments, and recovers automatically from anomalies during long-running operation.

No single ReAct loop can provide all four characteristics at once, so they have to come from the system architecture around it. Similar design constraints tend to produce similar architectures. When the goal is an always-online system that responds to environmental events concurrently, we expect personal agents and agents for commercial work such as software development to end up with similar designs. Building such a system also raises many problems much like those long studied in computer systems, so many classic design ideas from that field apply here too.

Implementations will vary, but we think five modules form the basic framework that supports these characteristics. The two-level architecture (Section 2) provides both *continuous* interaction and *productive* concurrency. The event-driven design (Section 3) makes the agent *proactive*. Full-duplex multi-channel interaction (Section 4) keeps each exchange with users both *continuous* and *productive*. Separating compute, state, and environments (Section 5), together with a Watchdog (Section 6), keeps the agent *persistently* online across devices and lets it recover from anomalies on its own.

The diagram below gives an overview of the Super-agent architecture. The modules labeled [2] through [6] are covered in Sections 2 through 6:



Overall architecture of a Super-agent, with labels [2] through [6] corresponding to Sections 2 through 6

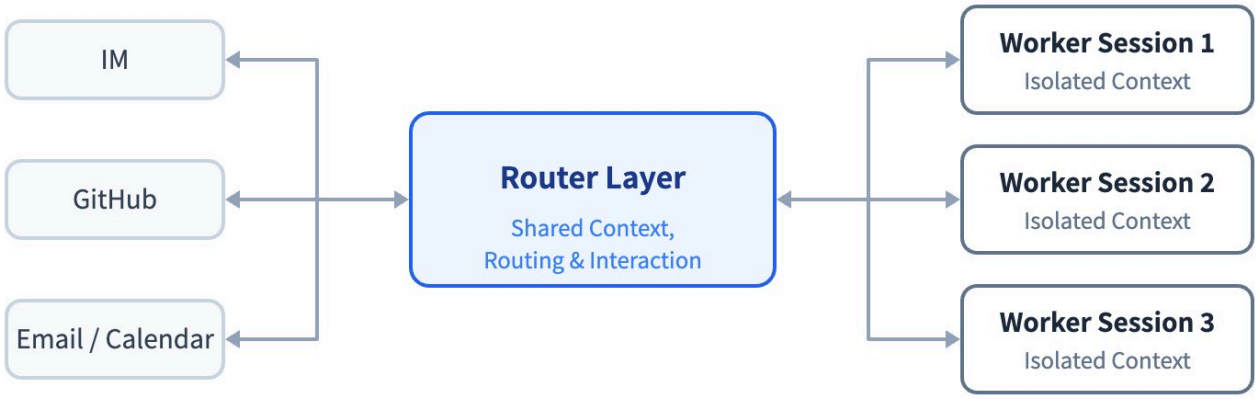
2. Two-Level Architecture

Supporting both *continuous* interaction across channels and *productive* concurrency across tasks forces a fundamental trade-off between continuity and throughput in how the system organizes its sessions.

Routing every event to one session gives the most continuous context, because that session sees everything. But a ReAct loop runs one step at a time, which caps how fast a single session can work: tasks take a long time and throughput stays low. The intermediate details of different tasks also end up interleaved in one context, which degrades the results.

Giving every event its own new session reverses the trade-off. It maximizes concurrent throughput, but when tasks are not fully independent, the lack of a shared global context leaves their results disconnected. Several sessions replying to the user separately also make the interaction confusing.

To get both continuity and throughput, we can split the work between two layers. A long-lived **Router layer** (Section 2.1) maintains global continuity by handling all human interaction and global coordination, while a context-isolated **Worker layer** (Section 2.2) runs tasks with high concurrency, as shown below:



Two-level architecture: the Router layer handles all routing and external interaction, while multiple isolated Worker Sessions execute tasks concurrently

With the two-level architecture, the system takes over context management and concurrent execution, so users seldom have to think about sessions and always work with the same Super-agent. Making users manage sessions by hand, then, is largely a design habit carried over from low-level LLM APIs. Sessions should be an implementation detail of the system, not a mental burden on

its users.

In existing designs, a session does two jobs: it manages the model's context, and it is the user's unit of project management. Once the product is decoupled from the underlying sessions, project management no longer has to follow session boundaries. What interaction and task management to offer users becomes a separate product question, which we return to later.

The two levels are a logical division of responsibilities. Neither layer has to be implemented as a single session, and each can have its own internal structure. The next two subsections describe the two layers in turn.

2.1 Router Layer

The Router layer is a long-lived global routing layer that receives human interaction from every entry point. All cross-task information and decisions come together there, so it serves as the system's global sequence point, coordination point, and memory. To keep this single global node from becoming a performance bottleneck, the Router layer is designed not to execute tasks itself. It only interacts with humans and dispatches tasks, and it can run without a Bash tool at all.

With one central node dispatching tasks to several execution nodes, this structure looks like a typical Agent Team, but the two focus on different things. An Agent Team usually breaks one complex task into steps and coordinates the roles that work on them. Because those steps tend to be tightly coupled, the Team Lead often has to understand the task in depth, plan subtasks, and coordinate members frequently, which adds up to a heavy reasoning load. In a Super-agent, the Router layer only dispatches tasks and tracks how they relate to each other at a high level. It does not break down individual tasks. Conversely, an Agent Team can also work as a two-level architecture. For that, its prompts must keep the main session from executing tasks itself and limit it to external interaction and to dispatching separate tasks to members, who work on them independently. In our experience, the Router layer's work is often simpler than expected, and a lighter model can usually provide both speed and quality.

Internally, the Router layer does not have to be a single session either, and some of its logic may not need an LLM at all. We chose the most direct design, a single long-lived LLM session, and found that it is less likely to become a throughput bottleneck than it might seem. The reason is that environmental events (the Triggers of Section 3) that arrive during one inference turn queue up and are processed together in the next turn. The result is **batching**, and processing the events together adds little overhead:

- When external events arrive at a steady rate, Triggers reach the Router Session one at a time, and each response comes back quickly.
- When events arrive in bursts and start to queue at the Router Session, the Triggers that built up during the previous turn are merged automatically and injected together into the next LLM call. Because the Router Session only does lightweight interaction and dispatch, each step usually has a long input and a short output. Triggers merged into one LLM API call are computed in parallel during Prefill, so the time per request grows much more slowly than the batch size. Up to a point, then, the larger the backlog, the lower the amortized inference time per decision. The limit depends on the context length and on how many Triggers the model can handle accurately in one turn.

Section 7.2 quantifies the effect of batching with an experiment. Combined with subscription and pass-through (Section 3.2) and Chain of Recall (Section 3.3), a single long-lived Router Session keeps the context continuous and is already enough for a team of our current size.

If the system grows to the point where one session really cannot keep up, the Router layer can use a multi-stage filtering structure internally, or be split into several sessions by **horizontal sharding**. Its internal design then comes down to the same trade-off between continuity and throughput. The finer the sharding, the more concurrent throughput the Router layer gets, but the less memory and context the shards share. This trade-off will remain until the global memory problem is solved.

2.2 Worker Layer

The Worker layer consists of the sessions that execute tasks. Each Worker has an isolated context and a full set of execution tools, and is dedicated to a single task. Worker Sessions directly determine the result of each task and the quality of its code, so they usually need the most capable models.

Workers never interact with humans directly. Their reasoning process and tool calls stay out of external channels. Instead, each Worker reports its status to the Router layer, which handles all replies to the user. This way, the system keeps both its concurrency and the quality of its output, without the confusion of several sessions replying separately.

When tasks run concurrently, each Worker works on its own task, and peer Workers usually do not need to communicate directly. Conflicts between Workers tend to arise in a shared working environment, such as the same Pull Request. As long as the Workers involved all subscribe to that environment's Thread, they receive the same event notifications (Section 3.2) and have a chance to spot and resolve the conflict on their own. The Router layer relays the few cases that need coordination across tasks.

Like the Router layer, a Worker does not have to be a single session internally. For a complex task, a Worker can create subagents or be an Agent Team itself, and this is transparent to the Router layer above it. The same structure fits code collaboration, where independent tasks such as reproducing issues, reviewing Pull Requests, and developing features can proceed at once without blocking each other. It also fits personal agent scenarios, even though a personal agent usually offers only a single chat page. When a user discusses several topics at once in that chat, the page keeps the conversation continuous. Meanwhile, the backend can dispatch a separate Worker for each topic to work on tasks and draft replies in parallel.

3. Event-Driven Triggers

To make a Super-agent *proactive*, we first need to change what drives the system. The agent's main input is no longer prompts that a human types into a session one at a time, but semi-structured events from the environment. In a Super-agent, these environmental events, such as group chat messages, emails, calendar reminders, code review comments, and CI results, are called **Triggers**. Because the agent listens for and responds to Triggers, it no longer has to wait for a human command before each run, as a single-session agent must.

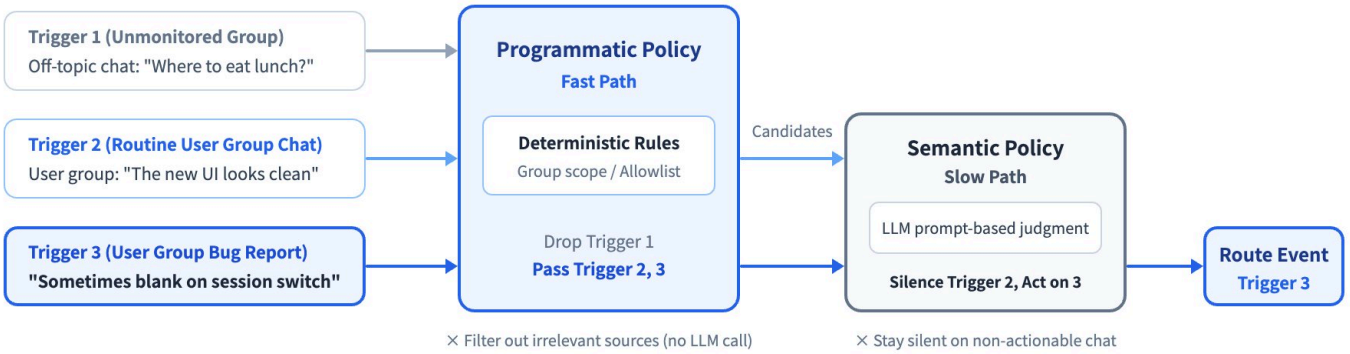
The subsections below follow an environmental event from arrival to processing, in four steps that build on each other. First, **two-stage policies** that combine code and models filter out the large volume of noise (Section 3.1). Next, the **subscription and pass-through** mechanism routes events to Workers (Section 3.2). **Inline & Spill and Chain of Recall** then inject dense context into the model and recall more of it on demand (Section 3.3). Finally, **speculative execution** lets the agent act ahead in the background based on environmental signals (Section 3.4).

3.1 Event Filtering

Once connected to real group chats and code platforms, the system sees a noisy event stream with a low trigger rate: the vast majority of everyday messages need no action from the agent. Having a large model judge every event from scratch would add too much latency and use too much compute. Borrowing the **Fast Path / Slow Path** idea from computer systems, we treat large-model reasoning as a flexible but expensive Slow Path. Frequent, well-defined, stable judgments can instead become deterministic code, which serves as a low-latency, low-cost Fast Path.

So the system filters every incoming event first. Deterministic policies remove noise before any large model is called, and only candidate events that need semantic understanding go to the model. We use **two-stage filtering** that combines programmatic policies with a large model:

- **Programmatic policies (Fast Path):** Code runs fast admission checks on deterministic conditions such as the event source, the group, and user or bot allowlists. For example, each source can be set to "respond only to explicit requests", "treat as candidate events", or "ignore". These checks drop most irrelevant messages before any model is called.
- **Semantic policies (Slow Path):** Candidate events that pass the programmatic checks go to the LLM, which decides whether to act based on the context and on policies written as prompts. The semantic stage also serves as a natural-language extension point, so humans can turn policies on or off, or add new ones, through prompts at any time, as shown below:

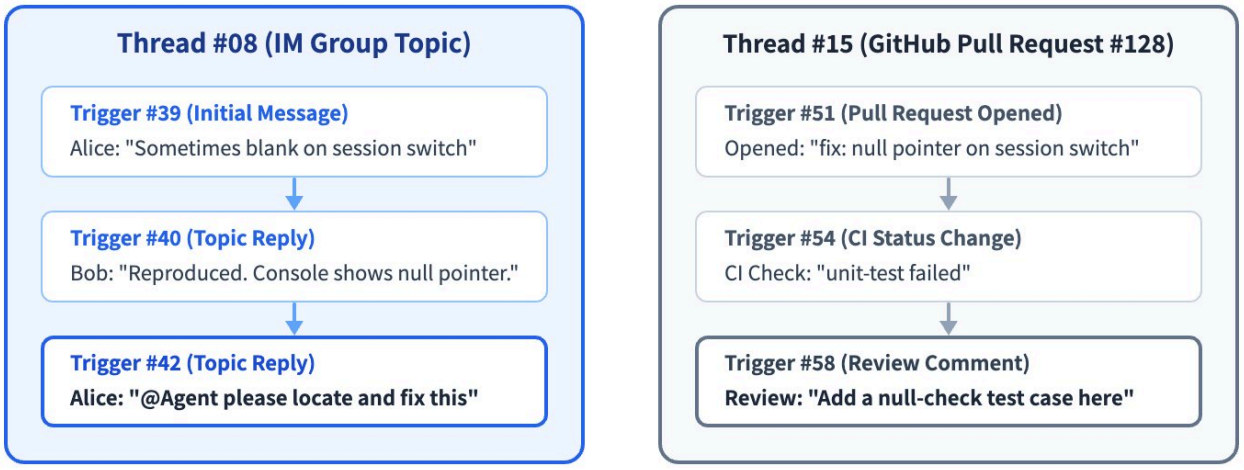


Two-stage filtering: programmatic policies first screen out most irrelevant messages, and semantic policies judge the rest

3.2 Event Routing

The global Router layer from Section 2 keeps interaction continuous across channels. But if every follow-up event for an assigned task still had to pass through the Router layer, this single global node would carry a heavy load and slow down responses. So the Fast / Slow Path split also applies to routing filtered events. The Router layer handles only cold starts of new tasks and cross-task coordination, while code sends follow-up events directly to the Worker Session that owns them.

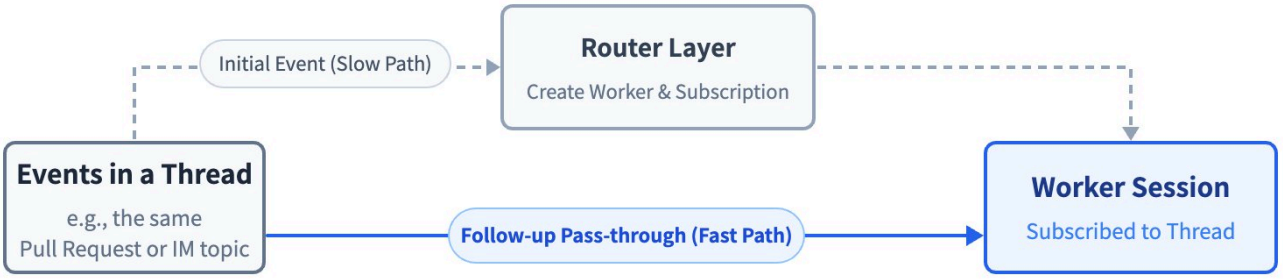
In a real workflow, consecutive events on the same GitHub Pull Request, IM topic, or email chain form an event stream, which we call a **Thread**:



Thread example: consecutive Triggers under the same IM topic or GitHub Pull Request form a Thread

When the Router layer creates a Worker Session for the first event in a Thread, that Worker subscribes to the Thread directly. From then on, event routing follows one of the two paths shown below:

- **Subscribed (Fast Path):** Code passes follow-up events in the Thread straight through to the subscribed Worker, so the Router layer does not need another tool call to forward them. If a new event falls outside the current task, the Worker can unsubscribe or hand the event back to the Router layer to be dispatched again.
- **Unsubscribed (Slow Path):** The Router layer can still see every event, which lets it track global state, but it only evaluates and dispatches new events that have no subscriber yet.



Subscription and pass-through: the initial event goes through the Router layer to create a subscribed Worker, and follow-up events in the same Thread go directly to that Worker

Subscriptions between Threads and Workers are many-to-many. A Worker can subscribe to several related Threads, and a Thread can have several subscribed Workers. When Workers overlap in the same working environment or discussion, each new event in the shared Thread passes straight through to all of its subscribers at once. The Workers then have a chance to learn what the others are doing and resolve conflicts themselves, without point-to-point communication. On the output side, Workers still report key conclusions to the Router layer, which handles all replies to the user and keeps global memory in sync.

3.3 Context Organization

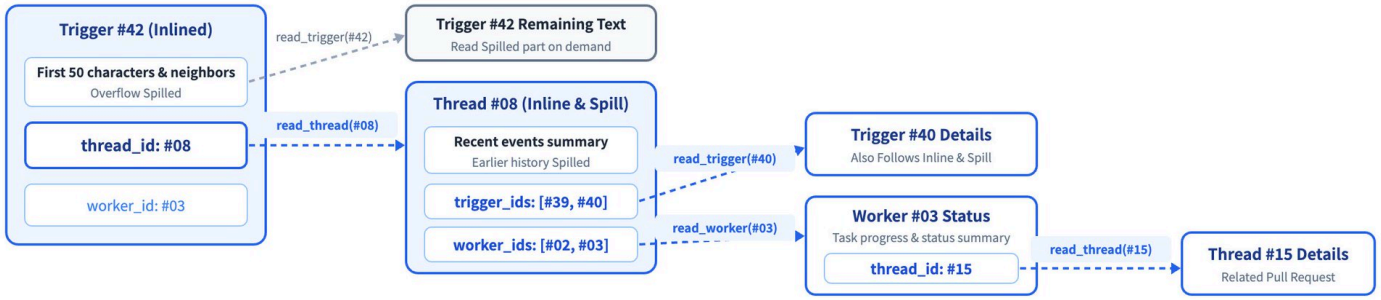
After an event has been filtered and routed to its target session, the next question is how much information to put into the model's context window. The input is no longer a limited number of human prompts but semi-structured events that the environment produces continuously, so the design of tool calls and event injection matters more. Inject too much, and the window fills up, diluting the model's attention. Inject too little, and the model has to call tools again and again to read context, which adds execution steps. Wasting the context window is wasting the model's intelligence.

We can therefore treat the limited context window as a cache. In terms borrowed from compiler optimization and computer systems, context injection comes down to two basic operations. **Inline** puts information likely to be needed soon directly into the current context window, saving extra tool-call round trips. **Spill** keeps full information outside the window when it exceeds the budget or is rarely used, and leaves only a reference or summary in the window, through which the model can read the full information on demand.

Combined with **locality**, a classic idea in computer systems design, **Inline & Spill** can improve event injection and tool calls step by step:

1. **Start with minimal information:** At first, give the model only the current object's ID and a summary of its content.
2. **Progressively Inline related information via locality:** Inline what the model is likely to need soon. Start with the IDs of closely related objects, then add content that is adjacent in space or time. Examples are the first 50 characters of the content, the IM messages around a message and the one it replies to, and neighboring comments in the same code review discussion.
3. **Spill content over the limit:** Set a small length cap for long content that cannot be inlined in full, and Spill the excess outside the window. A simple way to do this is to truncate the content and leave a pointer to where it continues, so the model can call a tool to read the rest when it needs to.

The Router layer has a context problem of its own. Many follow-up events pass straight through to Workers, and past tasks build up over long-running operation, so the Router layer cannot keep every task's full history in its context window. Yet when a user brings up an old task in another channel, the Router layer still has to recover that task's context. Applying Inline & Spill consistently to every kind of object in the system gives an on-demand, multi-hop retrieval mechanism: **Chain of Recall**. It reduces how much earlier information the Router layer loses over long runs, as shown below:



Chain of Recall: each recalled object inlines the IDs of next-hop objects, letting context expand hop by hop along references

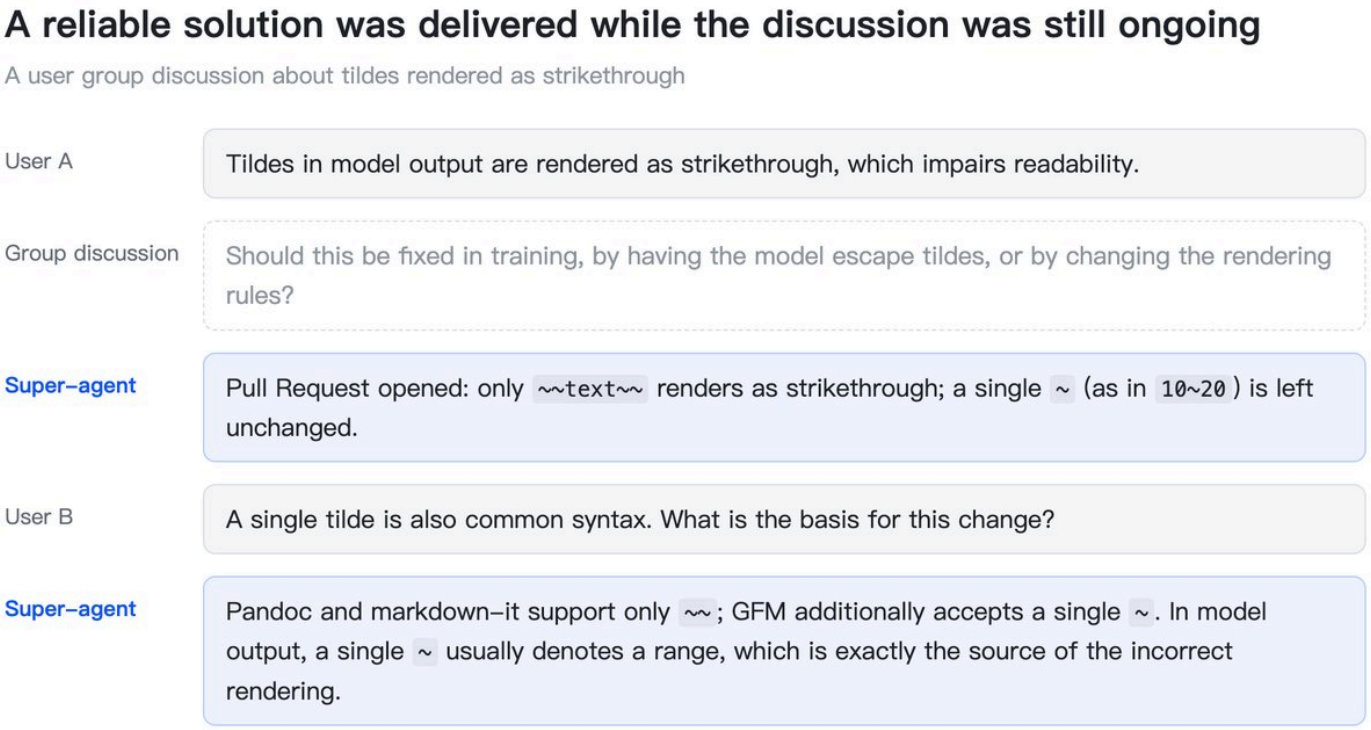
The system gives objects such as Triggers, Threads, and Worker Sessions unique IDs, and inlines the IDs of closely related objects when it injects one. These IDs link the objects into a reference graph. Each object recalled through a tool call also follows Inline & Spill, so the returned result inlines the summaries and IDs of the next-hop objects. Starting from the current event, the model can then follow these references and recall context hop by hop, as far as it needs.

For example, the model can start from the current message's Thread, follow an inlined ID to a related Worker Session, and then find another code review Thread linked to that Worker.

3.4 Acting Ahead

We also borrow **speculative execution** from computer systems. Based on environmental signals, the agent starts work in the background before it knows whether the result will be used. It uses extra background compute so that people wait less and need to step in less often. Results that turn out to be useful are delivered, and the rest are discarded.

In personal agent scenarios, when a meeting invitation email or a to-do message arrives, the agent can check for calendar conflicts, gather background information, and draft a reply in the background. The user then only needs to look over the draft and send it. The same approach works for R&D collaboration. In our internal DeepSeek Harness user group, for example, we configured a "silently reproduce and fix" policy. When someone reports an issue or discusses a proposed change, the agent first tries to reproduce the issue and find a fix in the background. It delivers a result only after it has confirmed the issue and found a simple fix. Otherwise, it stays quiet the whole time.



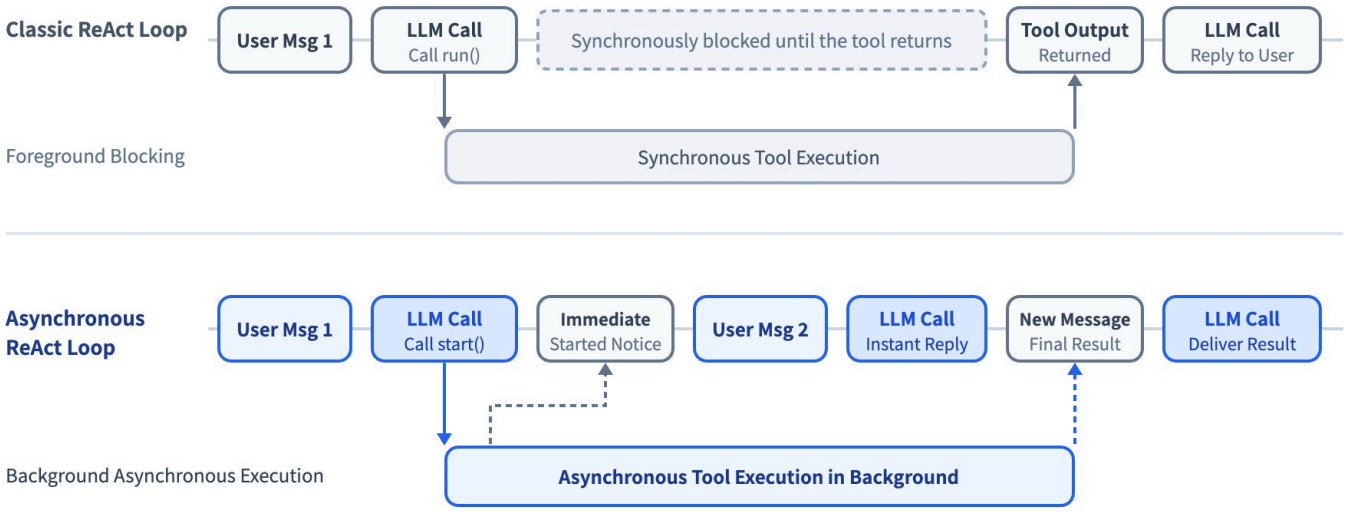
Speculative execution example: while the group is still discussing a solution, the Super-agent has already opened a Pull Request with a fix in the background

4. Full-Duplex Multi-Channel Interaction

The two-level architecture in Section 2 keeps interaction *continuous* across channels at the level of context. But to make day-to-day interaction both *continuous* and *productive*, two more problems need solving: how the interaction works and where it happens. The first is to keep time-consuming background operations from blocking the foreground conversation. An **asynchronous ReAct loop** solves this, so the session can accept new instructions at any time without waiting for a previous command or task to finish (Section 4.1). The second is to tailor the content and level of detail of the agent's responses to each communication channel and how people communicate there (Section 4.2).

4.1 Full-Duplex

In a classic ReAct loop, when the model makes a synchronous run() tool call, the loop blocks until the tool finishes and returns its final result. Until then, the session cannot respond to anything new from the user. To remove this blocking, the conversation has to move from a turn-based, half-duplex mode to a full-duplex one, as shown below:



Classic and asynchronous ReAct loops compared: an asynchronous call returns a Started notice immediately, and the session can still respond to new messages while background tools run

To do this, we replace time-consuming command and tool calls with asynchronous *start()* and *stop()* calls, turning the loop into an **asynchronous ReAct loop**. A call to *start()* returns "Started" immediately, and the loop does not wait for the tool. When the tool finishes in the background, the system delivers its final result to the session as a new event message. Based on the context, the model can also call *stop()* to end a background command that runs too long.

With the foreground conversation decoupled from background tasks, the agent can respond to people while time-consuming tasks run in the background. New input is handled as soon as the current inference step ends, rather than after background tools finish.

DeepSeek Harness already implements this asynchronous task mechanism, but it is not yet enabled by default.

4.2 Multi-Channel

People communicate differently in group chats, email, and GitHub discussions. The agent needs to adjust both the level of detail and the way it responds to suit each channel. Detailed reasoning and tool-call logs may suit a single-user terminal, but posting them unchanged in a group chat can overwhelm participants with information.

In a Super-agent, the two-level architecture already solves part of this problem. Each task's verbose reasoning process and frequent tool calls stay inside its Worker Session and never go straight to an external channel, while the Router layer handles all external interaction. Beyond that, the system needs to connect to the working environments users rely on every day and use prompts to set interaction norms for each channel:

- **IM and other instant messaging channels:** Keep communication light and interrupt people as little as possible. Like a human colleague, notify people only when a decision or sign-off is needed, ask only direct questions, and give short conclusions. One simple, effective hard rule we have verified is a cap of 100 words per reply.
- **Asynchronous channels such as GitHub, email, and documents:** Deliver structured, information-dense output. For example, when creating or updating a Pull Request, write out the full problem background, the change description, and the verification results according to the project's conventions. In personal agent scenarios, draft structured emails and document summaries directly.

5. Separating Compute, State and Environments

By default, local agents tie three things to one device: the **compute** that runs the Agent Loop, the session **state** that must persist, and the execution **environment**, such as code repositories and local toolchains. The agent stops whenever the device sleeps or goes offline, and it cannot reach environments on other devices. To keep a Super-agent *persistently* online and connected to the real

working environments its tasks need, we borrow **compute-storage separation** from computer systems and decouple the three in two steps. First, we separate compute from session state, which frees the agent from the lifecycle of any single machine and keeps it persistently online in the cloud (Section 5.1). Second, we decouple the cloud-hosted Agent Loop from state-heavy execution environments, which can then stay where they are while the agent reaches them across devices (Section 5.2).

5.1 Separating Compute from State

The compute that runs the Agent Loop can be stateless, and only the session state has to persist. So the first level of decoupling runs this compute in stateless, elastic cloud containers and keeps sessions and other persistent state in a database. The containers can scale out, restart, or be replaced, while persistent storage preserves the continuity of sessions and tasks. A local device that sleeps or goes offline no longer stops the agent, which stays persistently online in the cloud.

5.2 Separating Compute from Execution Environments

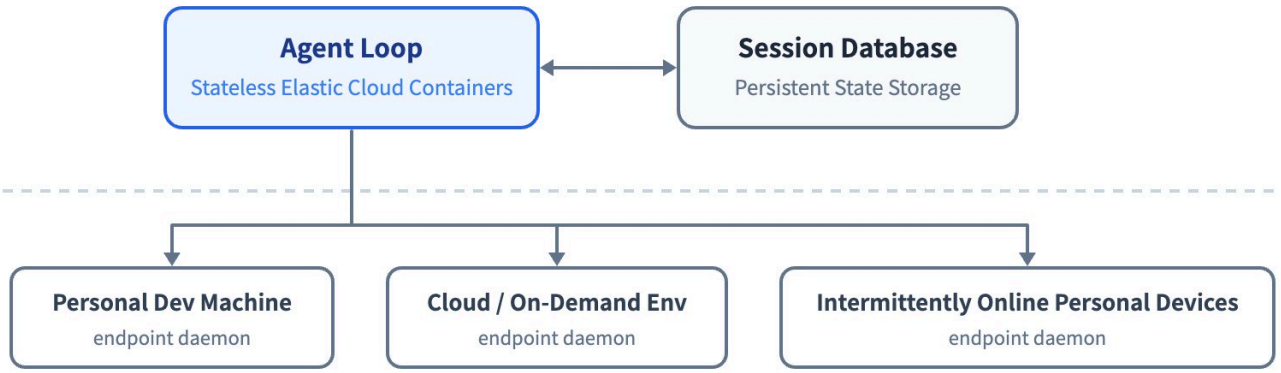
Once the Agent Loop and session state are in the cloud, the next question is how the agent reaches the execution environments its tasks need. Much of the context a model works with comes from tool calls against code repositories, local files, dependent services, and other execution environments. Broadly defined, then, context includes the execution environment itself.

There are two common approaches. The first starts a sandbox in the cloud, pulls the code, and tries to rebuild and sync the environment. But real working environments often include uncommitted local changes, private credentials, dependencies on intranet services, and specific hardware or toolchains. Syncing all of that is expensive, and it is easy to miss something. The second asks users to give up their local environments and move their work to the cloud, a migration that is too costly for many users.

Since the hard part to copy and sync is the environment itself, there is no need to move it to the cloud along with the Agent Loop. The second level of decoupling runs the Agent Loop in the cloud and leaves state-heavy execution environments where they are. Each endpoint needs only a lightweight daemon, which keeps no session state and does nothing but execute commands and return results. The execution environment stays outside the cloud, and the agent reads context from it on demand.

Anthropic's "Decoupling the brain from the hands" takes the same approach: the harness stays stateless, session logs are stored outside it, and execution environments are connected on demand through a unified interface. The difference is the target scenario. Anthropic's article describes a managed service and mainly discusses sandboxes created on demand and resources inside customer VPCs. We first need to connect the personal machines people use every day, which hold uncommitted changes and local credentials and may be online only intermittently. Our design therefore supports **BYOM (Bring Your Own Machine)**, which connects a user's own development machine or personal computer to the cloud agent. It also supports cloud-hosted machines and compute environments created on demand.

As a result, an agent is no longer tied to one machine. It can switch between execution devices, running commands and reading context in whichever environment a task needs. It can even work with devices that are only intermittently online: when a user's personal device reconnects, the cloud agent automatically starts and completes the tasks that need to run on that device. The diagram below shows the resulting structure, with compute, persistent state, and execution environments decoupled at two levels:



Compute-storage separation and decoupled execution environments: in the cloud, the Agent Loop is separate from persistent state and reaches different execution environments on demand through endpoint daemons

6. Watchdog

Staying *persistently* online takes more than the compute-storage separation in Section 5: the system must also recover from anomalies automatically during long-running operation. A persistently online distributed system rarely stays reliable for long if it relies only on its internal logic. It needs an independent **Control Loop** outside the main system.

Agent systems also need more than traditional process liveness probes or monitoring scripts. Many anomalies are not crashes: the process is still running, but the model is stuck in an unproductive loop or has drifted from its goal. A long-running agent system therefore has to monitor task progress as well as infrastructure liveness.

Our external Control Loop is an independent **Watchdog Session** that runs outside the main system and is driven by scheduled messages. The Watchdog handles infrastructure failures such as API errors and failed state recovery. It also uses the model to judge whether each Worker is making substantive progress or drifting from the overall goal. It fixes anomalies promptly and keeps rechecking transient external failures until they are resolved.

Because the Watchdog is itself an agent session with a full set of tools, maintainers can ask it in natural language why something failed. They can also have it correct the main system's behavior while the system runs, for example by adding or removing prompts or adjusting trigger rules.

7. Preliminary Experiments

We ran preliminary experiments on the core scheduling and routing designs at two levels: the whole system, and the Router layer on its own. The end-to-end experiment compares the two-level architecture with single-session designs and measures the effect of the subscription and pass-through mechanism (Section 7.1). The second experiment tests batching in the Router layer (Section 7.2). The experiments use synthetic workloads and simplified environments, and parts of the setup could be more rigorous, so the results below are indicative only.

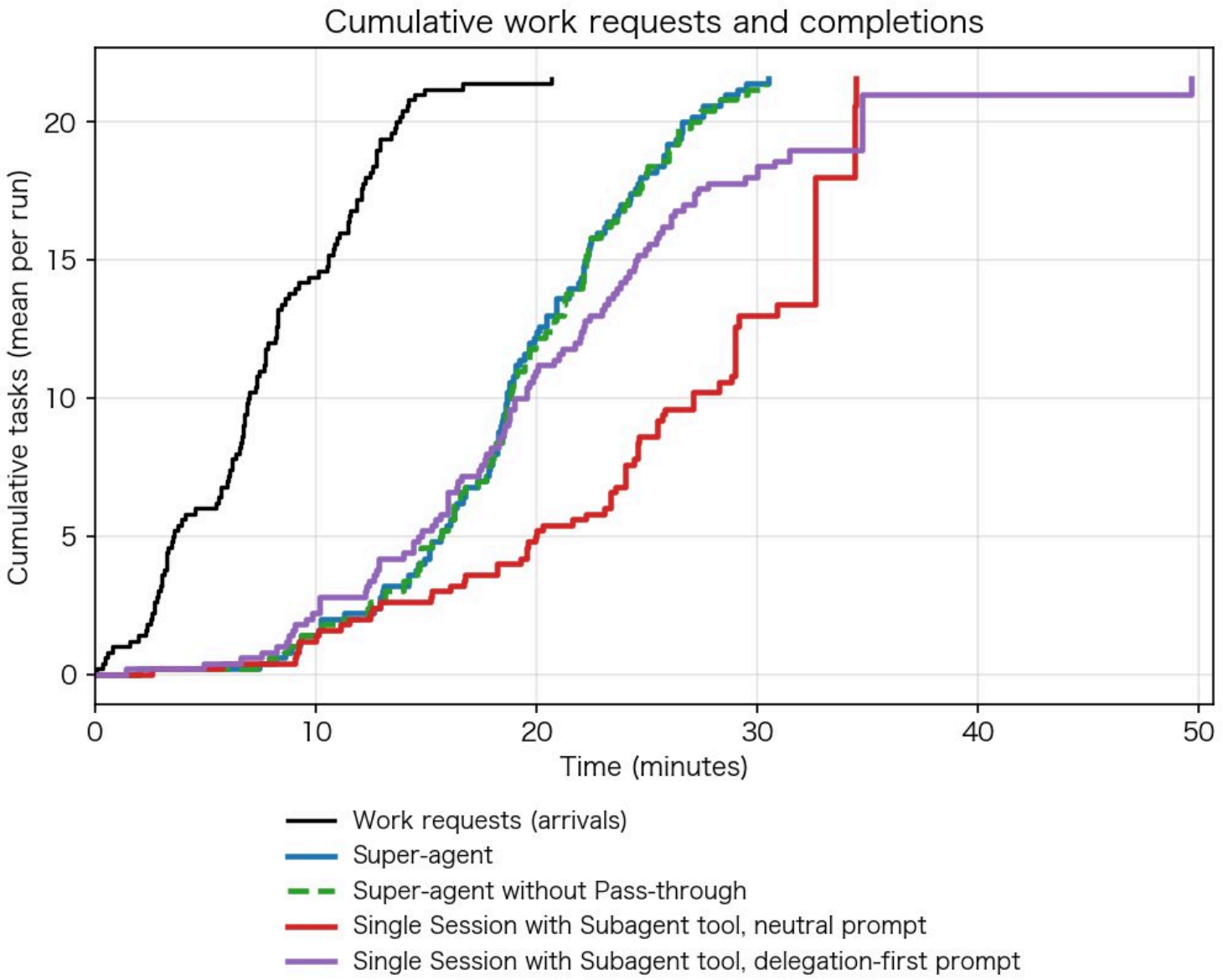
7.1 Ablation Study

Four end-to-end designs ran at the same time, driven by a stream of group chat and GitHub CI messages. Each run had about 300 messages, of which about 20 were work requests. The rest were casual chat, follow-ups, cancellations, and CI messages. Coding work was simulated by a tool that simply consumes time, so the experiment measures the system's scheduling and coordination, not its ability to execute individual tasks. Each design was run 5 times.

The four designs are **Super-agent**, **Super-agent without Pass-through**, and two versions of **Single Session with Subagent tool** that use different prompts.

The **neutral prompt** reads *"Use a subagent for a piece of work when appropriate..."*.

The **delegation-first prompt** reads *"Every message of the chat reaches you, so work you run yourself keeps stopping for new messages. Prefer handing each piece of work to a subagent of its own, and do only small things yourself...".*



Cumulative arrivals of work requests and cumulative completions under each of the four designs (mean of 5 runs; message intervals and work durations scaled by 1/4; model call times not scaled)

Design	Task completion time p50 / p95 (min)	Cache-read tokens per run (all sessions)
Super-agent	12.4/15.7	23.2M
Super-agent without Pass-through	12.6/15.5	31.8M
Single Session with Subagent tool, neutral prompt	20.4/29.5	6.1M
Single Session with Subagent tool, delegation prompt	13.8/29.0	10.8M

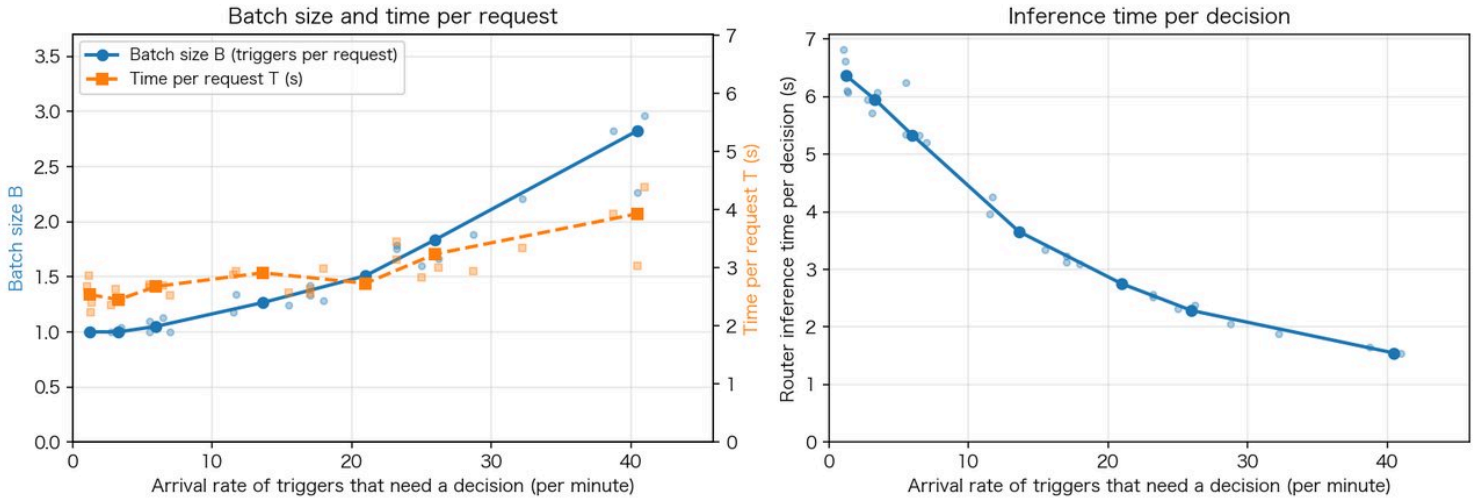
Note: each design was run 5 times. Task completion time is measured from a work request to its result message.

Super-agent completes tasks in 12.4 minutes at p50 and 15.7 minutes at p95, markedly faster than Single Session with Subagent tool. The single-session design is slower because, in this experiment, it depends heavily on its prompt: the main session often decides to do the work itself, which makes completion times long and unstable. With the neutral prompt, the model did not call a subagent in any of the 5 runs. All work ran in the main session, new messages kept interrupting it, and p95 reached 29.5 minutes. With the delegation-first prompt, the model delegated in 4 of the 5 runs. Completion times dropped somewhat, but overall they were still longer than Super-agent's. In the remaining run, the model judged the work "small" and did it itself, which produced the purple long tail in the figure. A single session has to rely on prompt engineering and the model's on-the-spot judgment to decide whether to delegate. The two-level architecture mitigates this problem at the system level instead: the Router does not execute work itself, and each Worker receives only messages from the Threads it subscribes to.

Because Super-agent maintains several independent sessions, it uses more cache-read tokens per run than the single-session designs, trading tokens for shorter and more stable completion times. Super-agent without Pass-through finishes tasks in about the same time as Super-agent, but its Router makes about 30% more decisions per run, and total cache-read tokens rise from 23.2M to 31.8M, about 37% more. So under this workload, pass-through does not make tasks finish faster or better, but it substantially cuts the Router Session's token use and inference overhead.

7.2 Router Layer Batching

To find out how much load a single long-lived Router Session can handle with the batching described in Section 2.1, we stress-tested the Router layer on its own. The Router called a real model, while the Workers used mock models. Step by step, we raised the arrival rate of Triggers that need a decision from about 1 to about 40 per minute. At each rate we measured the average batch size, the time per request, and the amortized inference time per decision. A decision can span several LLM requests, because the Router may reason over several steps or recall context through tool calls before it dispatches, so the time per decision can exceed the time per request.



Batch size, time per request, and amortized inference time per decision for the Router Session at different Trigger arrival rates

As the figure above shows, when the arrival rate of Triggers that need a decision rises from about 1 to about 40 per minute, the average batch size increases from 1.0 to 2.8. The time per request grows only from 2.5 s to 3.9 s, more slowly than the batch size, so the amortized inference time per decision falls from 6.4 s to 1.6 s. In other words, when external events arrive in bursts, the Router layer raises its effective throughput by automatically merging the events that have piled up. Batching also explains the result in Section 7.1: removing pass-through raised the Router's peak load, yet a single Router still did not reach its throughput limit. So at our current team size, there is no need to shard the Router layer prematurely at the cost of global context continuity.

8. Open Problems

Sections 2 through 6 describe the five modules that form the basic architecture of a Super-agent. The experiments in Section 7 show that its core scheduling and routing mechanisms work when many tasks run concurrently. The system can stay relatively simple largely because, at this stage, we have simplified both the engineering design and the scenarios we support. Beyond those

simplifications, problems remain open in four areas: scaling the two-level architecture (Section 8.1), working with an agent without sessions (Section 8.2), context boundaries and agent-to-agent (A2A) communication (Section 8.3), and security and privacy (Section 8.4).

8.1 How to Scale the Two-Layer Architecture

So far, a simple combination has covered most of our daily tasks at low engineering cost: a single long-lived Router Session, subscription and pass-through, and no direct communication between Workers. Once tasks grow larger and more tightly coupled than one Router Session can handle, the right structure for each layer is not yet settled. We still need to test more scenarios to find out when the Router layer should be sharded and how finely, and when peer Workers should communicate directly and in what collaboration topology. Co-designing the system and the model is another direction worth exploring. Training the model specifically for Router-layer work, such as judging events, dispatching in batches, and recalling context, may further improve the Router layer's throughput and decision quality.

Memory also limits how far the system can scale. The long-lived Router layer, together with Chain of Recall over the reference graph, mainly handles on-demand retrieval of recent tasks and session history. Once the system has run for months or longer, long-term memory becomes the main bottleneck.

8.2 How to Work with an Agent without Sessions

Because the two-level architecture hides multi-session management from users, sessions stop being the main concern of product design. But once the execution sessions underneath no longer map directly to conversations in the interface, a new question arises: how to present tasks to users, and what forms of interaction and project management feel natural. We think the chat window became the center of interaction partly through path dependence. More interaction may move into domain-specific interfaces, as with our Super-agent instance, which works directly in GitHub Issues and Pull Requests. Sessions can also remain in the interface because users are used to them, but one conversation the user sees need not correspond to one underlying session.

The earlier sections mostly cover human interaction at the start of a task and at sign-off. But during this transition, while models are improving quickly, many tasks still need a person partway through. It is also worth exploring how product design can let users intervene step by step on key details without losing the efficiency of autonomous background work. For example, users could step in, take over, or work with the agent in real time at any point during background execution.

Overall, the project management and interaction that users need depend heavily on their work domain and personal style, so the final design is likely to be highly customized. DeepSeek Harness has an open architecture, which makes such customization easier to support.

8.3 How to Define Context Boundaries and Collaborate via A2A

So far we have discussed the design of a single Super-agent instance and assumed that all information within its scope is gathered in one global context. As use grows from individuals and small teams to larger organizations, deciding how to provision Super-agents becomes, in effect, deciding which context is shared and which is isolated. Parts of an organization that need shared context share one instance, and parts that need isolation each get their own.

Once isolated instances exist within or across organizations, collaboration across their boundaries is A2A communication, for example between each member's personal agent and the team's shared Super-agent. Here A2A means agent-to-agent communication in general, not a specific protocol. We still need to work out how agents that share no underlying context and each represent a different permission boundary can communicate and collaborate securely and reliably.

8.4 How to Ensure Security and Privacy

The event-driven design in Section 3 lets a wider range of environments drive the agent, but it also exposes the agent to more untrusted input. Unless security and privacy are handled well, a Super-agent can only work within a very small, trusted scope.

For example, the design in Section 5 runs the Agent Loop in the cloud and installs lightweight daemons on execution devices. It implicitly assumes that the cloud and the endpoints trust each other. But the cloud-hosted Agent Loop keeps taking in external events, while endpoint devices hold private code, sensitive data, and local credentials. Connecting the two creates significant security and privacy risks. If a server-side intrusion or a prompt injection compromises the Agent Loop, every endpoint device connected to it could be taken over.

Credentials can leak even if the cloud is never compromised. As long as credentials on an endpoint can enter the model's context, normal execution can expose them. The agent therefore needs a way to use credentials without ever seeing them. For this architecture to be used more widely, it has to solve these security and privacy problems while keeping its ability to recall context from many different environments.

9. Conclusion

Rather than proposing new theory, this design applies classic engineering ideas from computer systems to building agent systems. Specific implementations will change as the technology develops, but we believe the system design ideas behind them will remain useful for longer. Our internal Super-agent instance, built on these ideas and on DeepSeek Harness, is already running continuously. It helps every day with reproducing and fixing issues, responding to code reviews, and developing DeepSeek Harness itself.

As we noted at the start, large models need less and less human intervention on individual tasks, so the bottleneck to further productivity gains is shifting to human attention. Together, the two-level architecture, event-driven design, asynchronous multi-channel interaction, compute-storage separation, and an external Watchdog let the system take over the tedious work of managing sessions and syncing context. People can then move further up, from execution details to higher-level decisions and sign-off.

Built this way, an agent is no longer only a chat window that waits for instructions. It becomes an autonomous collaborator that stays online in the digital environments people use. For individuals, it can be a personal agent that helps with daily tasks across devices. For teams, it becomes part of the R&D workflow, taking on tasks and moving them forward. We believe that as models and system architectures evolve together, this more autonomous way of working could become the norm in human-agent collaboration.

> *Parts of the design in this article are implemented in **dsh-superagent-prototype**. This plugin is only a prototype for validating the technical approach, not an official DeepSeek product.*

> *Author & Acknowledgements*

> *This article is based on shared work by the DeepSeek Harness team. Yujia Qiao implemented the system and wrote the article. Huanqi Cao, Zui Chen, Xieting Chu, Tianyi Cui, Shiyuan Feng, Yudong Han, Kexin Liu, Shichun Liu, Gesi Luo, Yifan Shi, Tongrui Xiong, Hanxiang Xu, C.C. Yu, Ziya Zhang, F. Zhou, and others took part in reviews and discussions and gave feedback. Both the system and this article owe much to the team's daily exchanges and the experience it has built up.*